

# Defense 4 — In-Network DNP3 Obfuscation on One Tofino-1: A Technical Explainer

RRC (size + CLRT) and BOR (bounded OPERATE release), validated against  
a physical SEL-751

DNP3 obfuscation project

Authoritative state: commit 5a0fb73 · branch  
defense4-size-native-parity-crc-split

## Contents

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| <b>1</b>  | <b>1. The problem being solved</b>                  | <b>2</b>  |
| <b>2</b>  | <b>2. The complete physical testbed</b>             | <b>3</b>  |
| <b>3</b>  | <b>3. DNP3 and TCP foundations</b>                  | <b>6</b>  |
| <b>4</b>  | <b>4. Defense 4 architecture</b>                    | <b>6</b>  |
| <b>5</b>  | <b>5. RRC in detail</b>                             | <b>8</b>  |
| <b>6</b>  | <b>6. BOR in detail</b>                             | <b>9</b>  |
| <b>7</b>  | <b>7. Why two scheduling domains were required</b>  | <b>10</b> |
| <b>8</b>  | <b>8. Packet-generation and reservoir mechanism</b> | <b>13</b> |
| <b>9</b>  | <b>9. Size splitting mechanism</b>                  | <b>13</b> |
| <b>10</b> | <b>10. P4 implementation walkthrough</b>            | <b>14</b> |
| <b>11</b> | <b>11. Control-plane setup</b>                      | <b>17</b> |
| <b>12</b> | <b>12. Hardware-only problems discovered</b>        | <b>18</b> |
| <b>13</b> | <b>13. Experiments and results (E0-E8)</b>          | <b>19</b> |
| <b>14</b> | <b>14. How to interpret every figure</b>            | <b>20</b> |
| <b>15</b> | <b>15. Claim and limitation guide</b>               | <b>23</b> |

**How to read this document.** Every section starts in plain language (“what and why”), then gives the exact technical detail. Numbers, code, and claims are taken from the frozen evidence at commit 5a0fb73, not from memory or summaries. Where the evidence only supports a partial claim, the text says so in the same breath. Figures referenced as *FIG-n* are in `./figs`.

# 1. The problem being solved

**Plain version.** A control center (“master”) talks to a protective relay (“outstation”) using DNP3 over TCP. Even if the payload is unreadable, someone watching the wire can recognize *which device* it is and *what it is doing* just from the shape and timing of the packets. That is called *fingerprinting*. Defense 4 makes one relay’s traffic look like a fixed policy instead of its own tell-tale self — without changing any DNP3 content.

**What the traffic looks like.** The master on TCP port 20000 sends a **READ** and the SEL-751 answers with a **RESPONSE**. The SEL-751 is a *separate-ACK* device (Case A): it first sends a pure TCP ACK, then — a moment later — the DNP3 response as a separate TCP segment. That small gap is measurable.

**How an observer fingerprints the device.** Three features leak, none of which require reading DNP3 payloads:

1. **Response size.** The SEL’s Class-0 response is a 49-byte DNP3 frame; on the wire it shows as a single segment we write [49]. Different devices emit different sizes.
2. **Segmentation.** How the response is split across TCP segments is itself a signature.
3. **Transaction timing — CLRT.** Formby et al.’s *Cross-Layer Response Time* is the delay between the transport-layer ACK and the application-layer response:

$$\text{CLRT} = T_{\text{response}} - T_{\text{TCP-ACK}}$$

Because CLRT is set by the device’s internal firmware timing, it is a stable per-device signature. On our physical SEL-751 the native CLRT is ~1.3 ms for a READ and ~2.1 ms for a SELECT, with high variance — enough to tell transaction classes apart.

**The two Formby fingerprints — and how Defense 4 answers each.** Formby et al. (NDSS 2016, *Who’s in Control of Your Control System? Device Fingerprinting for Cyber-Physical Systems*) proposed **two** device-fingerprinting methods that exploit the static, low-latency nature of ICS networks:

- **(a) Cross-layer response time (CLRT)** — a *network* fingerprint: the response-processing delay between the transport-layer ACK and the application-layer answer (the equation above). It identifies a device’s firmware/stack timing.
- **(b) Physical operation time** — a *physical* fingerprint: how long the device takes to actually *perform a control operation* (e.g., an electromechanical relay closing an output), measured through the protocol as the delay from the OPERATE command to its confirmation. This electromechanical latency is a property of the physical device itself.

Defense 4 is built to answer **both**:

| Formby fingerprint          | What leaks                                 | Defense 4 primitive                                                                                                                                                  |
|-----------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (a) CLRT (network)          | firmware response timing on READ/SELECT    | <b>RRC</b> — normalizes CLRT to a fixed 4 ms and fixes segmentation                                                                                                  |
| (b) Physical operation time | the relay’s true OPERATE actuation latency | <b>BOR</b> — releases the OPERATE at a hidden $T_0+J$ and anchors ACK/echo to $T_0$ , so the actuation latency and the hold are both unobservable (anti-subtraction) |

This is the core framing: **RRC defeats the cross-layer (network-timing) fingerprint; BOR defeats the physical operation-time fingerprint.** Both run in-network, on one switch, without changing a DNP3 byte.

**What Defense 4 protects — and what it does not claim.** It **replaces** one relay’s size, segmentation, and CLRT signature with a fixed in-network policy, and it releases control commands on a hidden delay so the command timing cannot be read off the wire. It **does not** claim multi-device anonymity (only one SEL-751 was tested — this is *signature replacement*, not indistinguishability among devices), it **does not** encrypt or hide payloads (a full TCP reassembler still recovers the 49 bytes), and it **does not** demonstrate exactly-once control delivery on the relay-facing wire (that link was not captured). §15 states the full boundary.

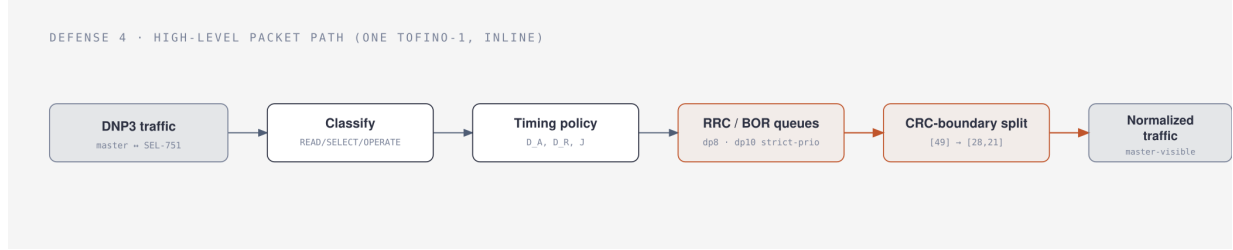


Figure 1: **FIG-0.** Defense 4 high-level packet path: DNP3 traffic is classified, a timing policy is applied, the packet is steered into the RRC or BOR queues, the response is split at a CRC boundary, and the master sees normalized traffic.

## 2 2. The complete physical testbed

**Plain version.** One real master, one real programmable switch, one real relay, in a line. The switch does all the work; nothing is simulated.

### Topology.

Vision (master, dp9) — observed WAN —> Tofino-1 —> SEL-751 relay (dp64)  
 (one pipe)  
 internal loopbacks: dp8 (RRC), dp10 (BOR)  
 internal pktgen/recirc/clone: dp68

- **Master** — host *Vision*, drives the DNP3 master. Campaign address 192.168.10.1, ingress on switch port **dp9**. (In the socket-harness lab the master/relay live on the 10.10.54.x LAN; the physical-relay campaign uses the 192.168.10.x addresses recorded in VERDICT.json.)
- **Switch** — one physical Intel **Tofino-1**, running the program `defense4_rrc_bor_unified12` (binary sha 33fa3a77). Ingress pipeline budget: **12 MAU stages, one pipe.**
- **Relay** — the physical **SEL-751** feeder-protection relay at 192.168.10.7:20000, DNP3 link address 0 (master = 1). It stays **READ-only** except in the one authorized, guarded SBO gate; its 32 binary outputs are monitored and stay OPEN throughout.

**Exact port map (the logical names used everywhere).**

| Logical    | dev_port | Role                    | Queues (strict priority, high→low) |
|------------|----------|-------------------------|------------------------------------|
| <b>dp9</b> | 9        | master ingress (Vision) | —                                  |

| Logical     | dev_port | Role                                        | Queues (strict priority, high→low)                                |
|-------------|----------|---------------------------------------------|-------------------------------------------------------------------|
| <b>dp8</b>  | 8        | RRC internal loopback (scheduling domain 1) | qid7 ACK_BLOCK > qid6 ACK_HOLD > qid5 RESP_BLOCK > qid4 RESP_HOLD |
| <b>dp10</b> | 10       | BOR internal loopback (scheduling domain 2) | qid3 OP_BLOCK > qid2 OP_HOLD                                      |
| <b>dp64</b> | 64       | relay-facing egress (SEL-751)               | qid0 default forward                                              |
| <b>dp68</b> | 68       | internal pktgen / recirc / clone-mirror     | — (not a host tap)                                                |

- **dp8 (RRC loopback)** recirculates a READ/SELECT response so it can be carved and held. Its four queues pace the ACK and the response independently.
- **dp10 (BOR loopback)** is a *second, independent scheduling domain* that holds the OPERATE. It exists precisely so the OPERATE-hold queues are not starved by RRC’s higher-priority reservoirs (see §7).
- **dp64 (relay egress)** carries the actual SEL-751 leg.
- **dp68 (pktgen/recirc/clone)** is where the on-chip packet generator injects blocker/token packets and where clones are mirrored. **It is an internal port, not a cabled tap** — this is the reason the relay-facing internal release is *inferred*, not captured (§12, §15).

**Why two internal loopbacks are still a one-switch inline defense.** dp8 and dp10 are not cables and not separate devices — they are switch-internal recirculation ports. All logic lives in the single Tofino pipe; the loopbacks only give RRC and BOR their own TM egress schedulers. The master and relay speak end-to-end through one box.

**Observation and threat-model locations.** The *attacker* is a passive observer on the master-facing link (dp9 side) — it sees exactly what the master sees. Every “observable” claim in this document is a **master-facing** measurement. The relay-facing link (dp64) and the internal release timing (dp68) are *not* part of the attacker’s view and were not instrumented.

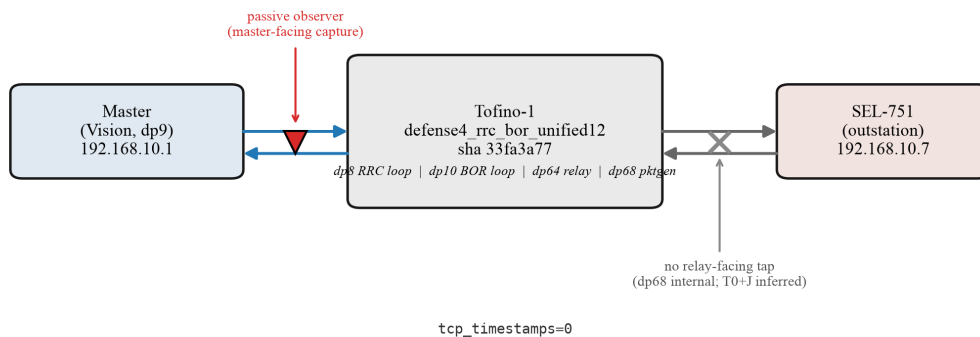


Figure 2: **FIG-7.** Physical testbed: Vision master → one Tofino-1 → physical SEL-751, with the internal loopbacks dp8 (RRC) and dp10 (BOR).

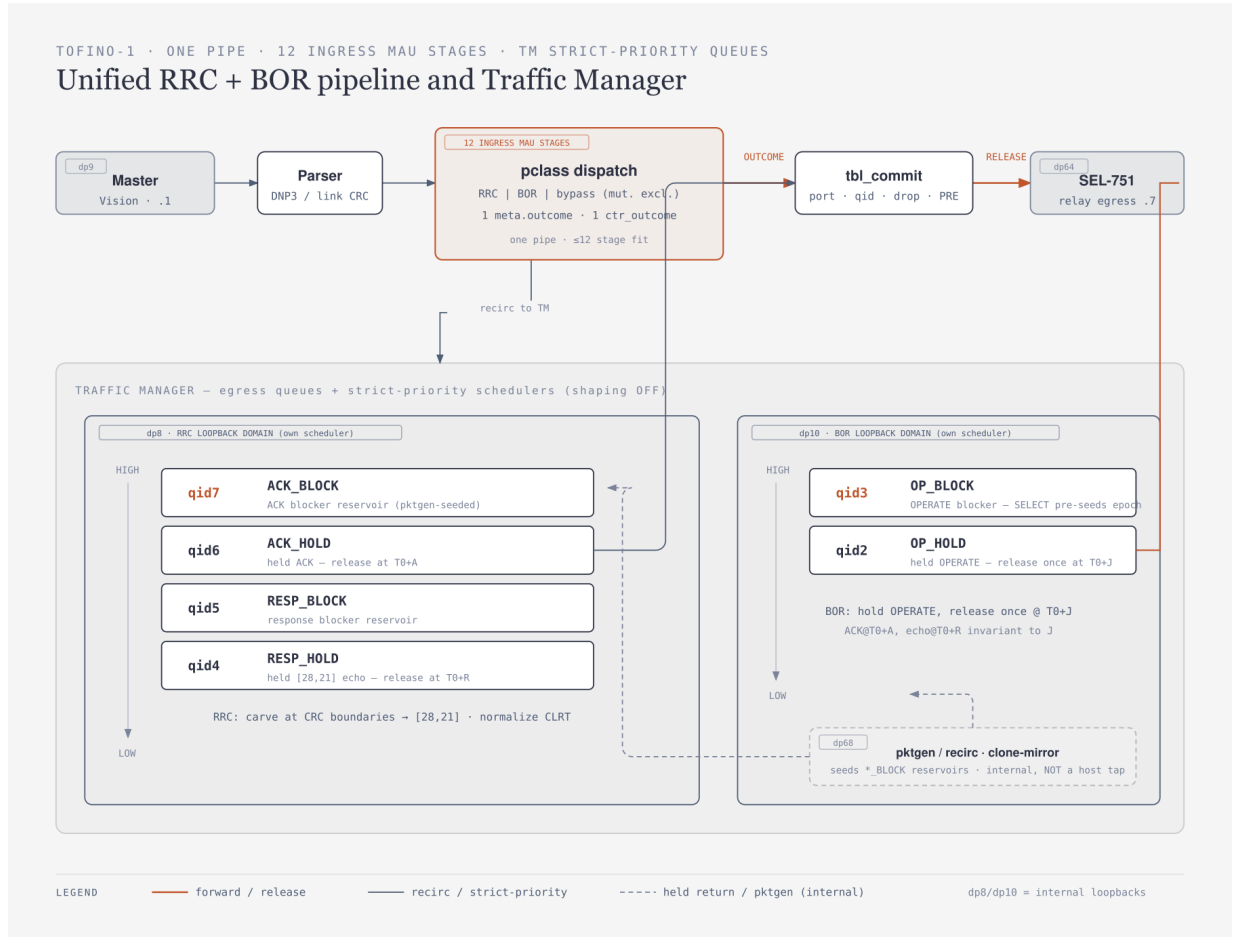


Figure 3: **FIG-8.** The Tofino-1 pipeline and Traffic Manager. Ingress parses and classifies, collapses to one meta.outcome and one tbl\_commit; the TM holds the six strict-priority queues — dp8 (qid7 ACK\_BLOCK > qid6 ACK\_HOLD > qid5 RESP\_BLOCK > qid4 RESP\_HOLD) and dp10 (qid3 OP\_BLOCK > qid2 OP\_HOLD) — with dp68 pktgen seeding the \*\_BLOCK reservoirs; egress on dp64 to the relay.

## 3 3. DNP3 and TCP foundations

**Plain version.** DNP3 is the language; TCP is the envelope. To understand the defense you need a few facts about both, and why you can cut a response into pieces safely but must not edit a byte.

**DNP3's three layers.**

- **Link layer** — the outer frame: start bytes 0x05 0x64, a length, control, destination and source link addresses, and a header CRC. The user data after the header is chopped into **16-byte blocks, each followed by its own 2-byte CRC**. These block boundaries are fixed by the standard and are the only places a stream may be cut safely.
- **Transport layer** — one byte (FIN/FIR/sequence) that lets a long application message span several link frames.
- **Application layer** — the request/response itself: an application control byte, a **function code**, and object data.

**READ, SELECT, OPERATE, and Select-Before-Operate (SBO).**

- **READ (0x01)** → the outstation replies with a **RESPONSE (0x81)**.
- A control is two steps: **SELECT (0x03)** arms a control point and the outstation echoes it back; **OPERATE (0x04)** then actuates. This **SBO** handshake means a stray or replayed OPERATE with no matching prior SELECT must not act. BOR is built around this: it admits exactly one OPERATE per armed SELECT.

**DNP3 link confirmation vs TCP ACK.** These are different things. A DNP3 link confirmation is an application-visible DNP3 frame; a **TCP ACK** is a transport-header acknowledgement invisible to DNP3. **CLRT is measured from the TCP ACK**, not the DNP3 confirmation — so the defense operates on the TCP-visible timeline.

**TCP sequence numbers and retransmissions.** TCP bytes are numbered by *sequence number*. If a response is emitted as two segments, the second segment's sequence number must continue exactly where the first ended, so the receiver reassembles the original byte stream with no gap or overlap. A lost segment is *retransmitted* — which is why BOR must recognize a retransmitted OPERATE and not actuate twice (§6, §12).

**Why a 49-byte response appears as [49].** The SEL's Class-0 response is a 49-byte DNP3 frame that fits in one TCP segment, so the observer sees a single 49-byte data segment: **[49]**.

**Why an arbitrary split would be unsafe.** Each DNP3 16-byte block ends in its own CRC. If you cut *inside* a block, a receiver validating block CRCs sees a broken frame. RRC therefore cuts **only on a completed block boundary** — for the 49-byte SEL frame, after the link header + block 0 (28 bytes), leaving block 1 (21 bytes): **[28, 21]**, with `join([28],[21]) == [49]` byte for byte and every block CRC still valid. No DNP3 field is modified and no CRC is recomputed.

---

## 4 4. Defense 4 architecture

**Plain version.** Defense 4 is the *unified* engine that folds three earlier timing defenses into one program and adds size normalization. A packet comes in, the switch decides what kind of transaction it is, applies a timing policy, steers it into the right hold queues, splits the response at a CRC boundary, and lets the master see uniform traffic.

**The complete packet path** (FIG-0, FIG-8):

DNP3 traffic → classify transaction → timing policy → RRC or BOR queues → CRC-boundary split → master

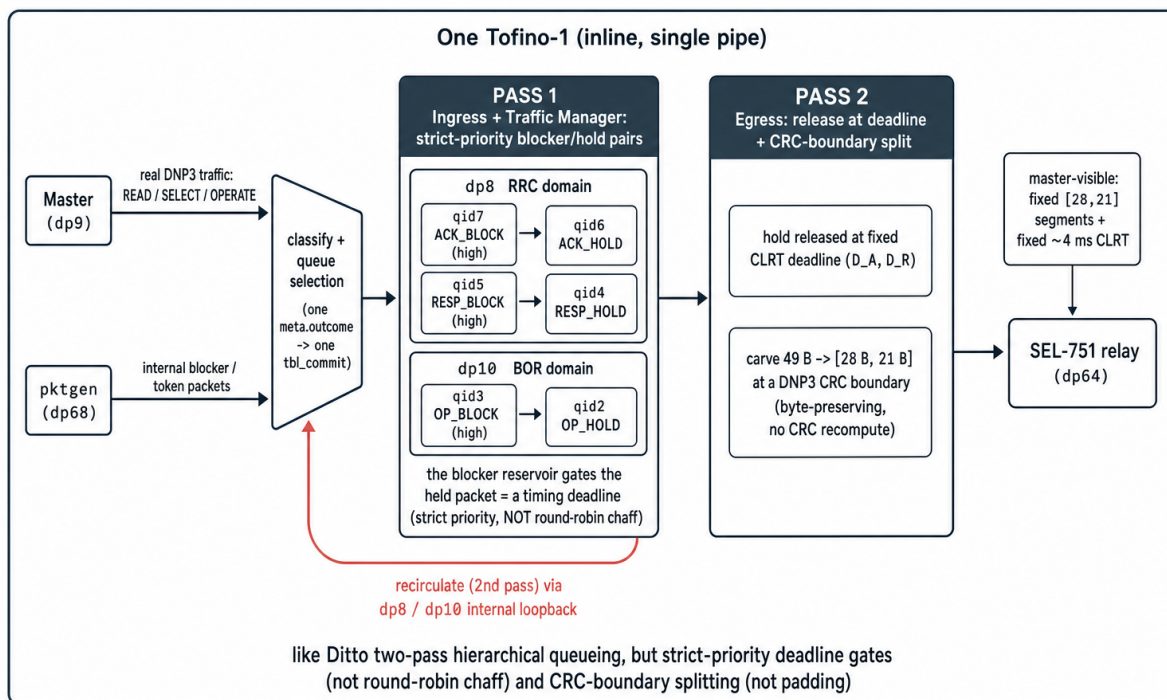


Figure 4: **FIG-13.** The Tofino-1 as a two-pass loopback pipeline. Real DNP3 traffic and internal blocker/token packets are classified into one meta.outcome/tbl\_commit; pass 1 places each held packet behind a higher-priority blocker reservoir (strict-priority *deadline gates* on dp8 RRC and dp10 BOR domains); the packet recirculates (2nd pass) and egress releases it at the CLRT deadline and carves 49 B → [28,21] at a CRC boundary. This mirrors Ditto’s two-pass hierarchical queueing but uses strict-priority deadline gates instead of round-robin chaff, and CRC-boundary splitting instead of padding.

**Relationship to the earlier defenses.** Defense 4 is the superset:

| Earlier primitive                    | What it did                                            | Where it lives in Defense 4             |
|--------------------------------------|--------------------------------------------------------|-----------------------------------------|
| <b>Defense 1</b> — event timing      | normalize <i>when</i> events appear                    | subsumed by the timing policy layer     |
| <b>Defense 2</b> — response deadline | hold the RESPONSE to a deadline                        | RRC’s RESP_HOLD (qid4), released at D_R |
| <b>Defense 3</b> — ACK deadline      | hold the pure TCP ACK to a deadline                    | RRC’s ACK_HOLD (qid6), released at D_A  |
| <b>Unified Defense 4 timing</b>      | one program, selectable modes + combined dual-deadline | defense4_rrc_bor_unified12              |
| <b>RRC response handling</b>         | recirculate + carve + hold (size + CLRT)               | dp8 ladder (\$5)                        |

| Earlier primitive           | What it did                                   | Where it lives in Defense 4  |
|-----------------------------|-----------------------------------------------|------------------------------|
| <b>BOR OPERATE</b> handling | admit one, hold, release at $T_0 + J$         | dp10 ladder (§6)             |
| <b>Size normalization</b>   | $[49] \rightarrow [28, 21]$ at a CRC boundary | egress split / deparser (§9) |

The two deadlines  $D_A$  (ACK) and  $D_R$  (response) are what make CLRT a constant: since  $CLRT = D_R - D_A$ , holding *both* the ACK and the response to fixed offsets pins the gap the observer measures.

## 5. RRC in detail

**Plain version.** RRC takes a READ or SELECT response, sends it around an internal loop, cuts it at a CRC boundary into a fixed shape, and releases it at a fixed time — so both the *size/segments* and the *CLRT* the master sees are policy values, not the relay’s own.

**The dp8 strict-priority ladder** (four queues, high  $\rightarrow$  low):

| qid | name       | role                                                     |
|-----|------------|----------------------------------------------------------|
| 7   | ACK_BLOCK  | reservoir of blocker packets that gate the held ACK      |
| 6   | ACK_HOLD   | the relay’s pure TCP ACK, held here                      |
| 5   | RESP_BLOCK | reservoir of blocker packets that gate the held response |
| 4   | RESP_HOLD  | the carved response, held here                           |

**How a queue-resident packet acts as a timing gate.** A held packet (ACK\_HOLD or RESP\_HOLD) sits in its queue behind a *higher-priority* reservoir of **blocker** packets (ACK\_BLOCK / RESP\_BLOCK). Strict priority means the blockers always leave first, so the held packet cannot dequeue until the blockers ahead of it are gone. The blockers drain on a schedule; when the reservoir for that transaction empties (blocker expiry), the held packet is released. This is how a *deadline* is realized purely by queue scheduling — no per-packet timer in the dataplane.

**Why the ACK and response deadlines are independent.** ACK\_HOLD (qid6) and RESP\_HOLD (qid4) have their own blocker reservoirs (qid7, qid5). The control plane sets  $D_A \approx 20$  ms for the ACK and  $D_R \approx 4$  ms for the response *relative to the ACK*, so the two release times are set independently and  $CLRT = D_R - D_A$  is exactly the configured constant.

**The RRC timing model.**

$$T_{ACK} = T_0 + D_A, \quad T_{response} = T_0 + D_A + D_R, \quad CLRT = D_R$$

(measured from the ACK), and on silicon this lands at **CLRT = 4.001 ms, std 0.02** for both READ and SELECT — replacing native 1.27 ms / 2.11 ms with high variance.

**Transaction matching, token isolation, cleanup, fail-open.**



- **Transaction matching** — a session table keyed on the reverse 5-tuple recognizes which flow a packet belongs to, in both directions.
- **Token isolation** — each transaction’s blockers are its own; the reservoir is pre-seeded by an on-chip packet generator (§8) so one transaction’s gate cannot affect another’s.
- **Cleanup** — once a held packet is released the transaction’s state is retired.
- **Fail-open** — if state is missing or a deadline cannot be armed, the packet is **forwarded unmodified** rather than dropped. The terminal commit table’s default action is a transparent forward (cmt\_fwd), never a drop (§10, §12).

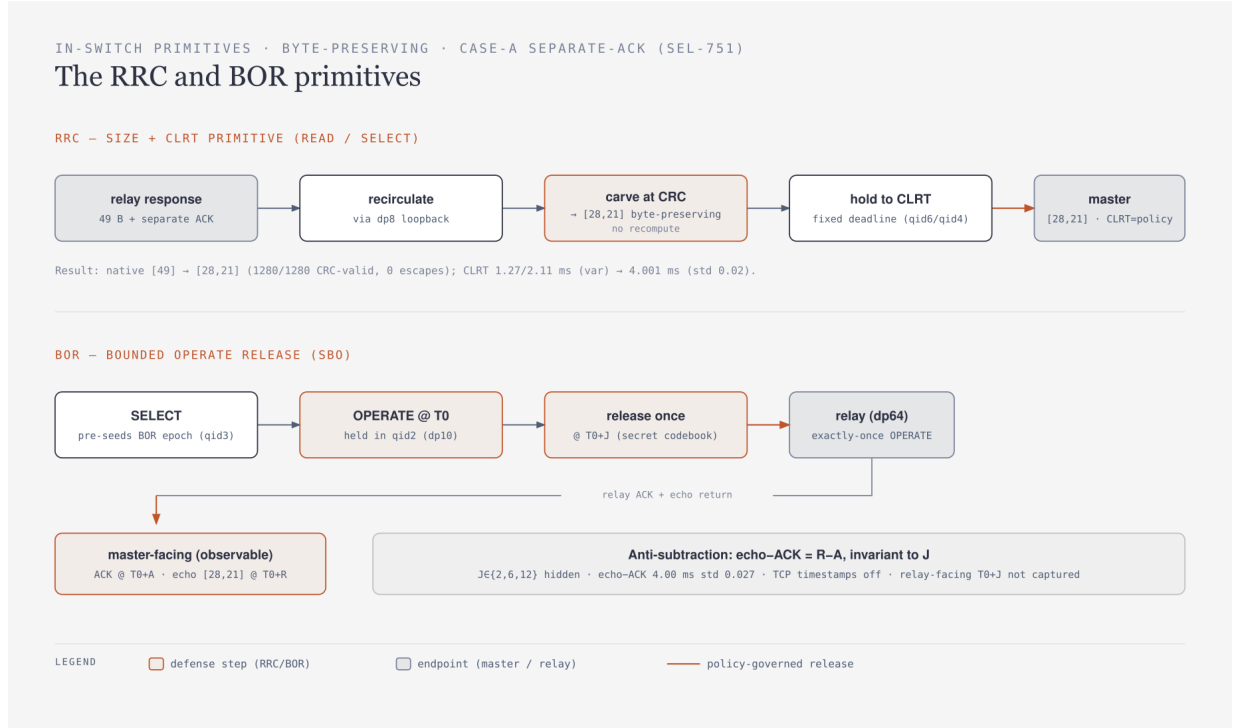


Figure 5: **FIG-10.** The RRC and BOR primitives as process flows: RRC recirculates → carves at a CRC boundary → holds to a fixed CLRT; BOR pre-seeds an epoch at SELECT → holds the OPERATE → releases once at T0+J, with ACK/echo anchored to T0.

## 6. BOR in detail

**Plain version.** BOR handles the one dangerous thing — a control command. It lets exactly one matching OPERATE through, holds it inside the switch, and releases it once at a *secret* delay, while making sure the master sees the ACK and echo at fixed times that reveal nothing about that delay. And if the OPERATE is retransmitted, BOR does not fire twice. This is what defeats Formby’s **physical operation-time** fingerprint (§1): an observer who times the OPERATE→confirmation gap learns the configured policy offsets A and R, never the relay’s true actuation latency or the hidden J.

### A complete SELECT → OPERATE transaction.

1. **SELECT reaches the relay normally** and, on the way through, *pre-seeds a BOR epoch* — it arms the OPERATE blocker reservoir (qid3 on dp10) so the hold is ready *before* the OPERATE arrives. The epoch is a fresh per-transaction marker, not the DNP3 sequence.

2. **The master sends OPERATE.** The switch **admits exactly one** matching OPERATE for that epoch.
3. **The OPERATE is held** in qid2 (OP\_HOLD) on the dp10 ladder (qid3 OP\_BLOCK > qid2 OP\_HOLD) — a second scheduling domain, so it cannot starve or be starved by RRC's dp8 queues.
4. **A policy-controlled J chooses the release delay.** J is drawn from a bounded secret codebook (admissible set {2, 4, 6, 8, 10, 12} ms), selected in-dataplane by a random byte so the observer cannot predict it from public header fields.
5. **The OPERATE is released once at T<sub>0</sub> + J** to the relay on dp64.
6. **ACK and echo deadlines stay anchored to the original T<sub>0</sub>.** The ACK is released to the master at T<sub>0</sub> + A and the echo (carved to [28,21] by RRC) at T<sub>0</sub> + R, both pinned to the OPERATE's own ingress timestamp T<sub>0</sub> — *not* to the later relay ACK.
7. **Retransmissions meet transaction state, not a second operation.** A retransmitted OPERATE after release reads the *spent-epoch* marker, is classified a duplicate, and is dropped.
8. **State is cleaned up after completion** — the epoch is retired; only the next SELECT starts a fresh one.

**Intended equations (distinguished from what was measured).**

$$T_{\text{OP,out}} = T_0 + J \quad (\text{intended; relay-facing, not captured})$$

$$T_{\text{ACK}} = T_0 + A, \quad T_{\text{echo}} = T_0 + R, \quad T_{\text{echo}} - T_{\text{ACK}} = R - A$$

**The anti-subtraction property.** Because A and R are anchored to T<sub>0</sub> and are *independent of J*, the observable difference  $T_{\text{echo}} - T_{\text{ACK}} = R - A$  is a constant. An attacker cannot subtract the ACK time from the echo time to recover the hidden J.

**What is hardware-supported vs unobserved.** On silicon, the master-facing invariance is **proven**: A ≈ 21 ms, R ≈ 25 ms, and **echo – ACK = 4.00 ms (std ≈ 0.026)** across J = 2, 6, 12 ms (the startup ACK maxima grow with J — 23.5 / 25.6 / 27.6 ms — and this is reported, not hidden). But **T<sub>0</sub> + J and the relay-facing exactly-once release were not captured** (dp68 is internal): single delivery is *inferred* from the round-trip, not measured. §15 keeps this boundary explicit.

## 7 7. Why two scheduling domains were required

**Plain version.** This is the key design lesson. RRC and BOR could not share one set of priority queues: RRC's high-priority blocker packets would hog the scheduler and the held OPERATE would never leave on its own secret schedule. Splitting them onto two internal loopback ports — two independent schedulers inside the same switch — fixed it, and it still fit in 12 stages.

**Why one strict-priority scheduler fails.** BOR seeds ACK and response blocker reservoirs (the same qid7/qid5 machinery RRC uses) for the held OPERATE. On a *single* strict-priority scheduler, those higher-priority reservoir tokens would monopolize the port, and the OPERATE-hold queues (qid3/qid2) could never be served on their own T<sub>0</sub> + J deadline — the OPERATE would drift toward RRC's ACK/response release times instead of its intended release.

*A note on framing.* An earlier informal description said “changing queue priority broke SELECT completion.” The frozen record does **not** substantiate that phrasing; the documented reason is **reservoir starvation** of qid3/qid2 by RRC's higher-priority queues on a shared scheduler (RRC\_BOR\_PRIMITIVE.md). This explainer uses the substantiated form.

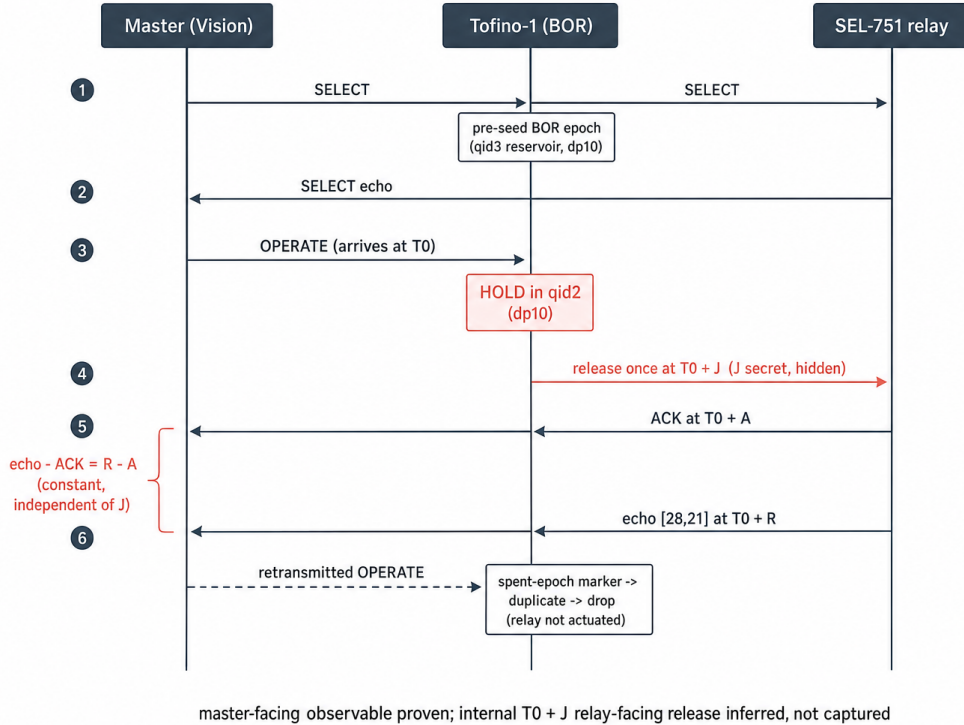


Figure 6: **FIG-12.** A complete SBO transaction. The SELECT reaches the relay and pre-seeds the BOR epoch (qid3 reservoir on dp10). The OPERATE arrives at T0 and is held in qid2; the switch releases it once at the secret T0+J. The master-facing ACK (T0+A) and echo (T0+R) are anchored to T0, so  $\text{echo} - \text{ACK} = R - A$  is constant and independent of J. A retransmitted OPERATE after release hits the spent-epoch marker and is dropped.

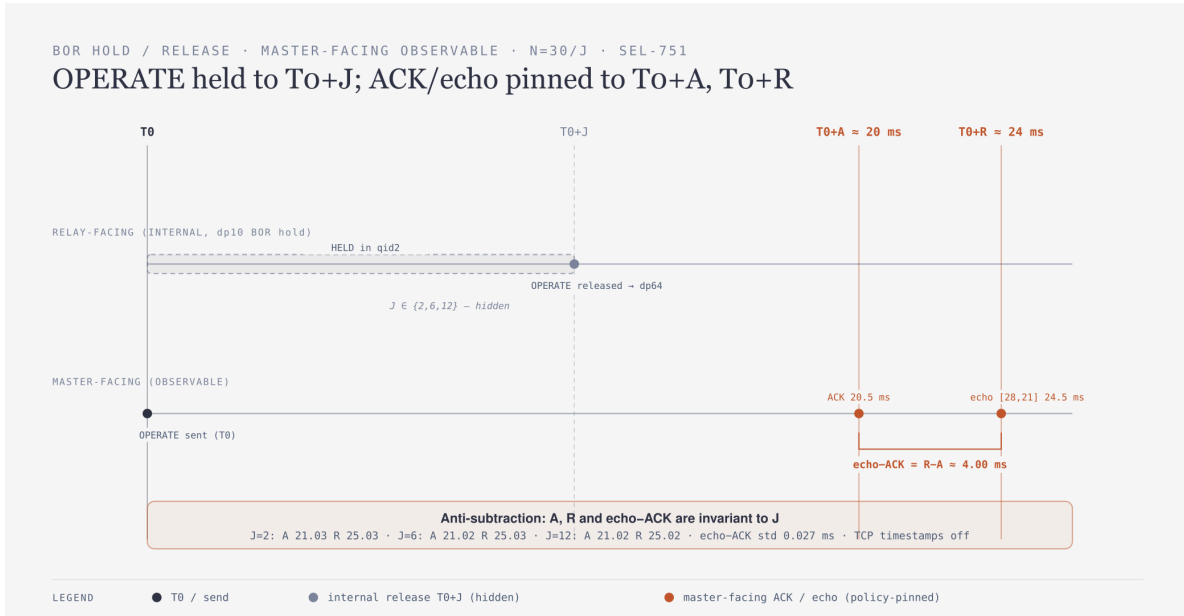


Figure 7: **FIG-9.** BOR hold/release timing and anti-subtraction on the master-facing observable: the internal release T0+J is hidden; ACK@T0+A and echo@T0+R are pinned to T0;  $\text{echo} - \text{ACK} = R - A \approx 4.00$  ms, invariant across J = 2/6/12 ms.

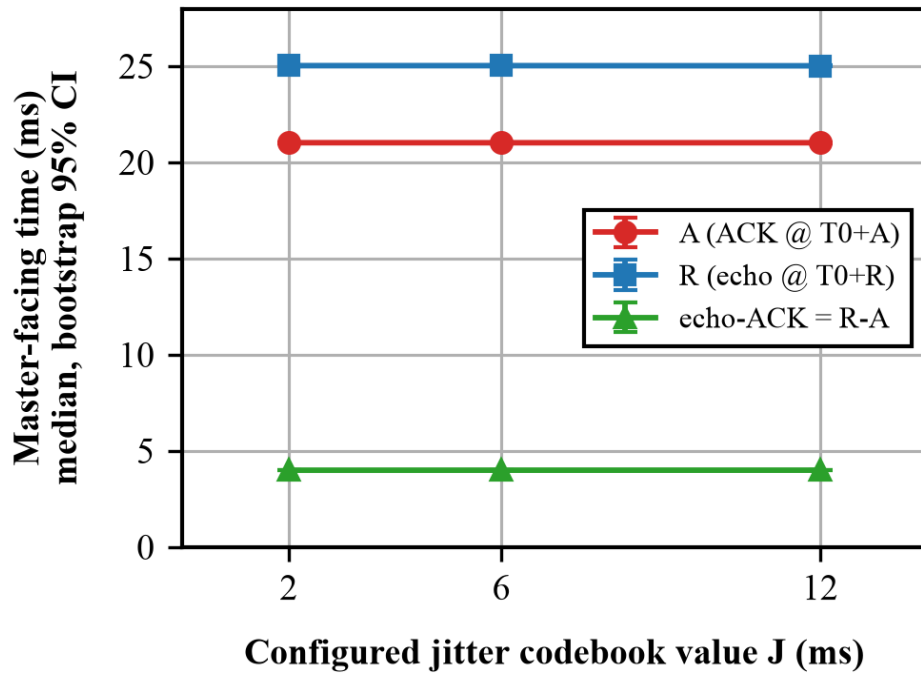


Figure 8: **FIG-4.** BOR J-independence: A and R stay flat and echo-ACK is a constant  $\sim 4$  ms bracket as J varies over 2, 6, 12 ms.

**How two domains fix it.** dp8 carries the RRC ladder ( $qid7 > qid6 > qid5 > qid4$ ) and **dp10** carries the BOR ladder ( $qid3 > qid2$ ) on its **own egress scheduler**. Now the OPERATE hold and the ACK/response holds arbitrate priority independently — neither can starve the other.

**Why it still fits in 12 ingress stages.** The two-domain split is a *Traffic-Manager* (egress queue/scheduler) construct — it costs **no** extra ingress MAU logic. The ingress fits because the match-action tail is flattened into one computed outcome and one commit (see §10). A register-domain separation (every BOR state access gated on a single `bor_pc` byte) keeps a `qid3` OPERATE token from ever touching RRC’s registers.

**Failed one-domain vs corrected two-domain (timeline).**

|                 | One shared scheduler<br>(failed)                     | Two domains dp8 + dp10<br>(shipped)         |
|-----------------|------------------------------------------------------|---------------------------------------------|
| OPERATE release | drifts to RRC’s response<br>time; $T0+J$ not honored | released at its own $T0 + J$                |
| ACK/echo anchor | coupled to shared drain                              | anchored to $T0$ (A, R<br>independent of J) |
| Ingress cost    | —                                                    | 0 extra stages (TM-only split)              |
| Result          | anti-subtraction broken                              | echo-ACK = 4.00 ms<br>invariant (silicon)   |

## 8. Packet-generation and reservoir mechanism

**Plain version.** The switch has an on-chip packet generator (pktgen). It does **not** make normal master/relay traffic — it manufactures the little “blocker” packets that sit in the hold queues and act as timing gates. Getting these blockers addressed to the right queue, for the right transaction class, was one of the harder bugs.

**What pktgen generates and does not.** pktgen emits internal **token/blocker** packets that pre-fill the block reservoirs (qid7 ACK\_BLOCK, qid5 RESP\_BLOCK, qid3 OP\_BLOCK). It never generates DNP3 requests or responses — real traffic always comes from the master or relay.

**The two pktgen profiles (2K vs 3K).** There are two applications on dp68, distinguished by a 32-bit pattern under mask 0xFFFF0000 (byte 0 = 0xE1, byte 1 = profile selector):

| App | name           | trigger pattern | packets   | seeds                        |
|-----|----------------|-----------------|-----------|------------------------------|
| 1   | 2K_operate     | 0xE1 00 * *     | 2·K = 128 | qid7 (ACK), qid5 (RESP)      |
| 2   | 3K_read_select | 0xE1 01 * *     | 3·K = 192 | qid7, qid5, <b>qid3 (OP)</b> |

with reservoir depth **K = 64**. The 2K profile arms the ACK+RESP reservoirs for an OPERATE hold; the 3K profile additionally pre-seeds the OP reservoir for a fresh READ/SELECT arm. increment\_source\_port = False is load-bearing (it keeps all K packets on one source port so the batch is not capped below 64).

**The clone-marker ambiguity that was found — and fixed.** Originally both the READ/SELECT fresh-arm path (cmt\_fwd\_clone) and the OPERATE-hold path (cmt\_op\_hold) wrote the *same* clone tag 0xE1000000. One marker cannot select between two pktgen apps, so the wrong reservoir could be armed. The fix uses **distinct markers in byte 1** — 0xE1000000 (2K) vs 0xE1010000 (3K) — and pins byte 0 **and** byte 1 in the trigger mask (0xFFFF0000), making the two apps mutually exclusive. The parser value set was expanded from one entry to two to match.

**Parser value-set entries.** A pgen\_recirc value set admits one byte per app, byte = (pipe < 3) | app\_id, mask exact 0xFF; with pipe 0 the admitted bytes are {0x01, 0x02} (app 1 and app 2). This is how a recirculated pktgen packet is recognized in the parser.

**Mirror / recirculation paths.** Clone/mirror **session 7** copies the triggering packet to dp68 (ingress mirror) so pktgen can re-inject the reservoir burst; PRE multicast group 0x2849 (nodes 0x2851/0x2852) fans the carved 49-byte echo back to the master on dp9.

**Reservoir population and expiration.** On a fresh arm the pktgen burst fills the reservoir to K = 64 blockers; as they drain (strict priority ahead of the held packet), the held ACK/response/OPERATE is released when its reservoir empties.

**Why an offline setup PASS was insufficient.** The control-plane setup has a full offline model and a fail-closed dry-run, but *“a compile is not silicon, and the dry-run readback is against an in-process model, not the switch.”* Several defects (§12) only appeared on hardware — a program can compile and pass its model and still be mis-wired on the ASIC.

## 9. Size splitting mechanism

**Plain version.** The switch turns one 49-byte response into two TCP segments of 28 and 21 bytes, cut exactly on a DNP3 CRC boundary, with the sequence numbers lined up so the

master reassembles the original 49 bytes. Nothing in the DNP3 message changes.

**Why [49] → [28, 21].** The 49-byte SEL response is: DNP3 link header + CRC (10 bytes) + data block 0 with its CRC (18 bytes) + data block 1 with its CRC (21 bytes). The only safe interior cut is on a completed block boundary. Cutting after the header + block 0 gives **28 bytes**; the remainder is **21 bytes**:

```
| link_hdr + hdrCRC | block0(16B)+CRC | block1(≤16B)+CRC |
|<----- 28 bytes (cut here) ----->|<---- 21 bytes ---->|
           segment 1                      segment 2
```

**How the segments carry contiguous sequence numbers.** The first segment keeps the original TCP sequence number; the second segment's sequence number is the first's + 28, so the two segments abut with no gap or overlap. The receiver's TCP stack concatenates them back into the original 49-byte stream. In the unified kernel this is done in **egress** without a mirror: ingress steers the response into an RRC multicast group (RRC\_MGID\_49\_28 = 0x2849) that produces two egress replicas distinguished by replication id — **rid 1** emits the 28-byte *prefix* (invalidate block 1 + trailer, total\_len -= 21, clear PSH+FIN), **rid 2** emits the 21-byte *suffix* (invalidate link header + block 0, tcp.seq += 28, total\_len -= 28); rid 0 (if any) is a byte-identical pass-through. Only the IPv4/TCP checksums are recomputed, per window.

**Why no DNP3 application modification is required.** The cut is on an existing block boundary, so join([28],[21]) equals the original 49 bytes byte for byte. No DNP3 field is edited and **no CRC is recomputed** — the block CRCs that were valid before the cut are still valid after it.

**How checksums and CRC validity were evaluated.** The verifier (size\_reconstruct.py) orders the relay→master segments by **TCP sequence number** (not arrival order — an earlier bug), reassembles the contiguous run from the 05 64 start, and validates: the segment vector, total = 49 bytes, sequence contiguity, the **DNP3 link-header CRC and each 16-byte block CRC**, and the **IP/TCP checksums** (rebuild-and-compare). Result at commit 5a0fb73: **1280 / 1280** defended responses are [28,21], all CRC-valid and checksum-valid, with **zero** 49-byte source-copy escapes.

**Why the claim is “CRC-valid reconstruction,” not “source-byte identity.”** No source-side original frame was captured as an oracle, so the evidence shows a **CRC- and checksum-valid 49-byte reconstruction** — not a byte-for-byte match to a captured source frame. This is stated plainly in VERDICT.json and honored throughout.

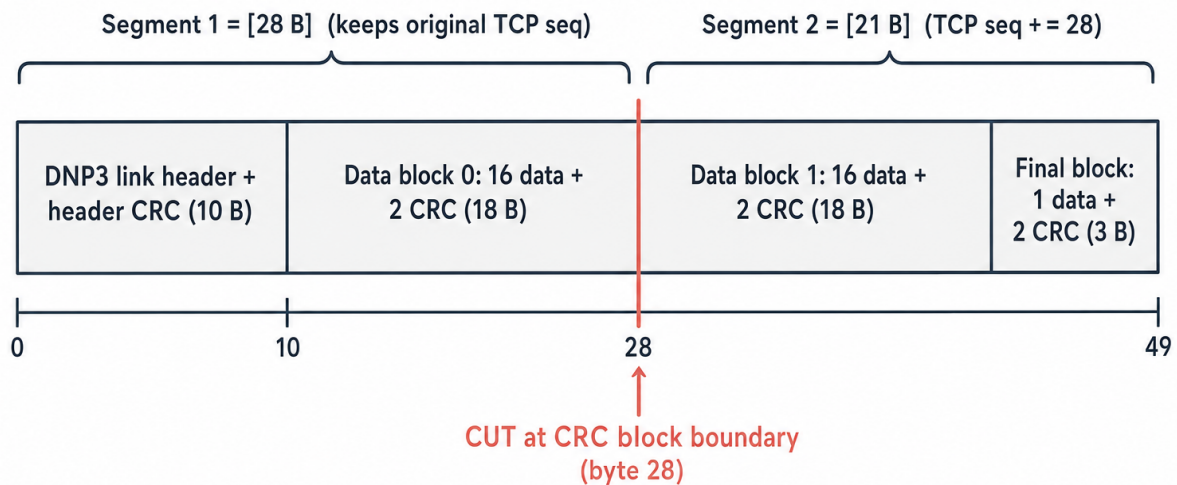
---

## 10 10. P4 implementation walkthrough

**Plain version.** The program is explained by logical section, not line-dumped. The single most important trick is that the whole match-action decision collapses into one computed outcome (meta.outcome) applied by one terminal table (tbl\_commit) — that is what fit RRC + BOR into 12 stages.

*(The facts below are read from the authoritative program defense4\_rrc\_bor\_unified12.p4, 3509 lines, at commit 5a0fb73, with line ranges where useful.)*

**Parser states.** ethernet → either an internal token frame (EtherType 0x88C1, forced to ROLE\_BLOCK) or ipv4 (TCP only, ihl == 5, fragments rejected) → tcp (a flags/offset gate that also sets the RRC 49-byte-eligibility bit payload49) → dnp3\_dl (link, LEN ≥ 8) → dnp3\_tp (transport, FIR & FIN) → dnp3\_app. A value\_set pgen\_recirc (size 2, exact 0xFF mask) recognizes generated-token leading bytes; dp68 clones are recognized by looking ahead for CLONE\_TAG\_BYTE = 0xE1.



byte-preserving: join([28],[21]) = original 49 B; no DNP3 field changed, no CRC recomputed;  
only IP/TCP checksums updated

Figure 9: **FIG-11.** The 49-byte SEL-751 DNP3 response and the cut point. The frame is a 10-byte link header (with its CRC) followed by CRC-protected data blocks. The only safe cut is on a completed block boundary: after the link header + block 0 (28 bytes), leaving block 1 + the final block (21 bytes). Segment 1 = [28], segment 2 = [21] with its TCP sequence advanced by 28; no DNP3 byte is changed and no CRC is recomputed.

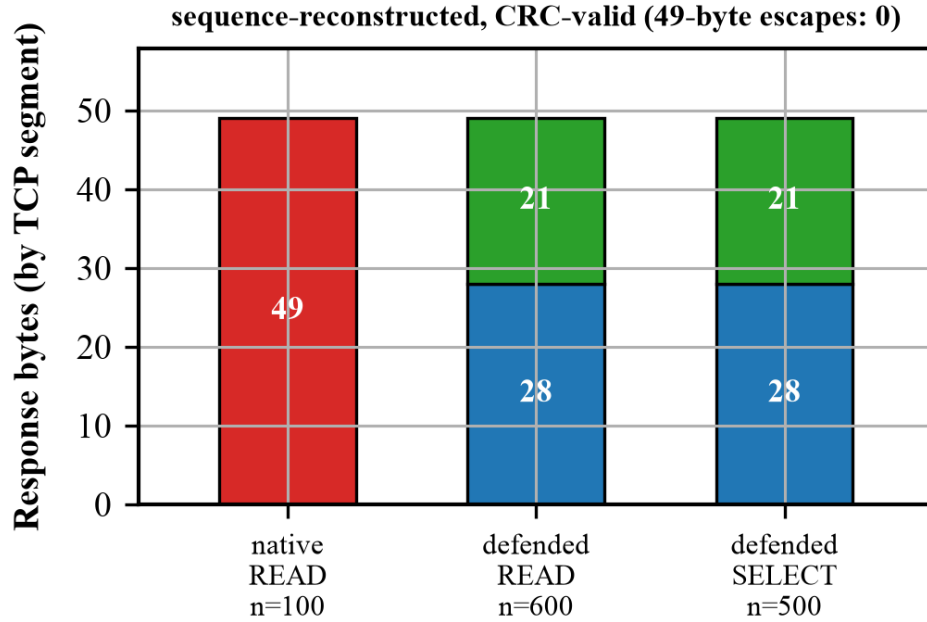


Figure 10: **FIG-5.** Segment-size distribution: native mass at 49 bytes; defended mass at 28 and 21 bytes, with zero 49-byte escapes.

**Metadata.** One ingress metadata struct carries the flow identity, timestamps (`ts32`, `now_word`), the timing params (`seq_m = D`, `mode`, `da_dr`), the deadline surface (`dl_val`, expired, and the RESPONSE-deadline twins), the verdict (`verdict`, `txn_active`), the flatten field **bit<16> outcome**, and — under the BOR build flag `U_BOR` — the BOR block (`bor_pc`, `epoch_stored`, `ready_stored`, `gen_stored`, `op_matched`, `hold_ok`, `rand8`, `j_ticks`, `a_ticks`, `r_ticks`, `clone_tag`, `clone_ses`).

**Transaction classification.** `READ` (0x01), `SELECT` (0x03) and `OPERATE` (0x04) are *folded into one role* `ROLE_ARM` at parse time; the `SELECT`-vs-`OPERATE` distinction is re-derived in the MAU by reading `func_code`, producing a single control byte `bor_pc`: `BPC_PREPARE` (`SELECT` arms the epoch), `BPC_OPERATE` (fresh `OPERATE`), `BPC_RELEASE` (a dequeued `OPERATE` at release), `BPC_TOKEN` (a blocker), `BPC_PKTGEN_OP` (a pktgen OP seed). Every BOR register access is gated on `bor_pc` so an `OPERATE` token can never touch RRC’s registers.

**Session table.** `tbl_session` matches the ternary 5-tuple in both directions (`sess_relay / sess_master`); session identity and expected sequence/ack are then *learned* in the dataplane (`reg_exp_relay_seq`, `reg_session_port`, `reg_exp_ack`) rather than pre-installed per transaction.

**BOR codebook.** `tbl_bor_codebook` is keyed on `hdr.tcp.dst_port` (exact, the flow/profile) and `meta.rand8` (range), with action `set_j(j_ticks)` and a **P4 default of `set_j(0)`**; `meta.rand8` is an in-dataplane PRNG draw (`Random<bit<8>> rng_bor_j`). The admissible set **{2,4,6,8,10,12} ms is control-plane data, not a source constant** — the setup installs it as non-overlapping `rand8` ranges over buckets 0..255 (validated total, no gaps/overlaps, none mapping to `J = 0`; ns words, low byte 0, `TICK = 256`). So `J` is a per-transaction PRNG draw the observer cannot predict from public fields.

**Decision-table flattening (the stage-shedder).** Instead of a deep tree of dependent action tables, two mutually-exclusive ternary tables (`tbl_decide_fresh / tbl_decide_deq`) are co-



placed in **one** stage and compute a single meta.outcome. This collapsed the 5-stage tail to *decide (1 stage) + commit (1 stage)*, taking lean RRC from 12 → **10** ingress stages; BOR then fit in the 2 freed stages, landing the whole unified program at **12 in a single pass**. The two-pipe / resubmit fallback was therefore **not needed**.

**Commit actions (tbl\_commit).** One terminal table keyed on meta.outcome, the *only* site that writes port / qid / bypass / drop / multicast. Its actions are cmt\_drop, cmt\_fwd, cmt\_fwd\_clone, cmt\_shape, cmt\_block, cmt\_resp\_block, cmt\_hold, cmt\_resp\_hold, and (under U\_BOR) cmt\_op\_block, cmt\_op\_hold, cmt\_op\_relay. Every OUT\_\* value is const-mapped single-valued, so outcome == 0 is unreachable and the **default action is a transparent forward cmt\_fwd — never a drop** (this repudiates Blocker 1, §12). A single DirectCounter ctr\_outcome on the commit replaces 36 scattered counter sites.

**Register / state domains.** The BOR domain is four registers — reg\_bor\_epoch, reg\_bor\_ready, reg\_bor\_gen (the dedup / exactly-once marker), reg\_bor\_topj (the T0+J deadline) — all zero-initialized fail-closed, every access gated on bor\_pc so an OPERATE token never reaches RRC's registers. The RRC timing core carries its own reg\_tag (held to exactly **4 RegisterActions**, the Tofino SALU cap), reg\_deadline, reg\_tresp, reg\_ack\_rel, the learned session trackers, and telemetry registers. This register-domain separation is what lets the two scheduling domains coexist in one pipe.

**Clone generation and queue assignment.** The fresh-arm path clones with tag 0xE1010000 (triggers the 3K app: seeds qid7/qid5/qid3); the OPERATE-hold path clones with tag 0xE1000000 (2K app: seeds qid7/qid5) and enqueues the OPERATE to qid2. Held packets are assigned: ACK → qid6, response → qid4, OPERATE → qid2, with their blocker reservoirs on qid7/qid5/qid3 respectively; egress port dp8 (RRC) or dp10 (BOR).

**Recirculated-packet recognition.** A packet returning on dp8/dp10 sets meta.dequeued = 1 from its *ingress port*, but the MAU then keys on meta.dequeued plus packet content (token vs release), **never on the port itself** — so the logic is port-independent. pktgen/clone packets on dp68 are recognized by the parser value set {0x01, 0x02} and the 0xE1 clone-tag look-ahead. The clone/echo itself is produced by an ingress-to-egress Mirror (clone\_mirror) on session 7 to dp68.

**Split / deparser and checksum handling.** The response carved to [28,21] is emitted as two TCP segments; the deparser sets the second segment's sequence number contiguously and the IP/TCP length and checksum fields are updated for each segment (no DNP3 byte touched, no DNP3 CRC recomputed).

**Fail-open paths.** On any miss/timeout/absent-state the disposition falls through to cmt\_fwd (unmodified forward). The design never drops on the default path — availability first.

*For each action, the pattern is: input (class + session + register state) → state read/written → output port + qid → the failure it prevents. The defect ledger in §12 lists the specific failures each guard was added to prevent.*

---

## 11 11. Control-plane setup

**Plain version.** Loading the program is only half the job; the control plane must bring up the ports, build the queues at the right priorities, load the pktgen apps, program the timing and J codebook, and — critically — read back every write and treat any mismatch (or warning) as a failure.

**Setup procedure (unified single-pipe build).**

1. **Port activation** — bring up dp8, dp9, dp64, dp68 (frozen D3 helper), then the second loopback dp10 directly: 25G, FEC none, autoneg force-disable, BF\_LPBK\_MAC\_NEAR, enabled; readback asserts loopback mode / speed / enable.
2. **Real port-group resolution** — a logical name  $\rightarrow$  (pg\_id, pg\_nr) resolution, then queue index pg\_queue\_of(pg\_nr, qid) (real silicon numbers, not the offline stand-in).
3. **TM queue configuration & priorities** — via tm.queue.sched\_cfg: max\_priority = qid, scheduling\_enable = True, **both rate-enables False (shaping OFF)**. dp8: qid7 > qid6 > qid5 > qid4; dp10: qid3 > qid2. Readback asserts strictly descending, distinct.
4. **PRE / multicast nodes** — RRC echo carve: group 0x2849, nodes 0x2851 (RID 1) / 0x2852 (RID 2), both fanning to dp9.
5. **pktgen applications** — two apps on dp68 (2K/3K, §8): pattern/mask, packets\_per\_batch = 127 / 191, increment\_source\_port = False, token templates at buffer offsets 0 and 256.
6. **Parser value sets** — pgen\_recirc admits {0x01, 0x02} (mask 0xFF).
7. **Mirror session** — session 7  $\rightarrow$  dp68 (ingress), for clone re-injection.
8. **Session entries** — tbl\_session reverse 5-tuple, both directions, device-scope 0xffff.
9. **Timing parameters** — D\_A = 20 ms, D\_R = 4 ms ( $\Rightarrow$  CLRT 4 ms); BOR A = 20 ms, R = 24 ms (ticks = ms  $\cdot$  1e6).
10. **J codebook** — admissible {2,4,6,8,10,12} ms partitioned over rand8 buckets 0..255, non-overlapping, validated total (no gaps/overlaps, none maps to J = 0).
11. **State initialization** — zero reg\_bor\_epoch/ready/gen/topj, read back, assert 0 (fail-closed).
12. **Readback verification** — every write is read back; Checks.expect fails on mismatch. An offline self-test deliberately corrupts a value to prove the comparator is fail-closed.
13. **Watchdog / rollback** — there is **no config snapshot**; rollback is a forward verified teardown (disable pktgen  $\rightarrow$  shaping off  $\rightarrow$  timing off  $\rightarrow$  delete PRE) to a benign forwarding state. The safe whole-switch restore baseline is the frozen Defense 4 timing build or Defense 2.

**Why configuration warnings are treated as failures.** In the unified build, Checks.blocked() returns true if there is **any fail or any warning**; pktgen and shaping are enabled only if nothing is blocked. If blocked, the mechanism is left **disabled** (fail-closed) rather than half-configured. Every hardware op also refuses unless DEFENSE4\_HW\_AUTHORIZED=1.

## 12. Hardware-only problems discovered

**Plain version.** A design that passes a compiler and an offline model can still fail on silicon. These are the concrete bugs the campaign found and fixed — they *are* part of the contribution, because they explain what “runs on hardware” actually costs. The central ledger is the six-blocker correction pass (RRC\_BOR\_UNIFIED12\_FIXED.md).

1. **Tables that compiled but were not wired.** The first “unified setup” was a placeholder with invented control-plane schemas — *“a compile is not silicon, and the dry-run read-back is against an in-process model, not the switch.”* A separate instance: the loaded .conf referenced a nonexistent model JSON, so the binary would not even load until a corrected conf was used.
2. **The dangerous tbl\_commit default (Blocker 1).** tbl\_commit had const default\_action = cmt\_drop() with no entries — loading before the control plane populated it would drop **every** packet. Fixed to a total const map (OUT\_\* 1..43) with a safe cmt\_fwd default.
3. **Missing OPERATE reservoir seeding (Blocker 2).** cmt\_op\_hold enqueued the OPERATE but never seeded its ACK (qid7) and RESP (qid5) reservoirs, so the relay’s ACK and

- echo dequeued immediately and *escaped* normalization. Fixed to arm the same pktgen clone burst the fresh-arm path uses (mutant `operate_no_seed` → killed).
4. **ACK-relative vs T0-relative deadlines (Blocker 3).** Deadlines were armed at the relay ACK (`t_ACK-relative`). Since BOR delays the OPERATE by J, an ACK-relative hold **leaks J** (the anti-subtraction attack). Fixed to anchor `reg_deadline = T0 + A`, `reg_tresp = T0 + R` to the OPERATE's own ingress T0; the later relay ACK cannot re-anchor them (mutant `ack_relative_deadlines` → killed). Confirmed on silicon: `echo - ACK = 4.00 ms`, independent of J.
  5. **Premature BOR-state clearing (Blocker 5).** Clearing `reg_bor_gen` at release let a retransmitted OPERATE read `GEN_INACTIVE` → fresh → forward to the relay a **second** time. Fixed to keep the generation as a **spent** marker until the next SELECT; a post-release retransmit reads `gen == spent` → duplicate → drop (mutant `clear_gen_at_release` → killed).
  6. **Clone-profile ambiguity.** One clone marker could not address two pktgen apps (§8). Fixed with distinct byte-1 markers and a size-2 value set; proven READ-neutral by bit-identical MAU placement.
  7. **Incomplete configure-all.** Configure-all did not bring up dp8/dp9/dp64 ports (added `config_ports`) and was missing timing args (`--d-a/--d-r/--poll-ms`). (`read_len` is a retired D3-only field carried as 0 — a no-op here, not an active defect.)
  8. **Pipe-0 symmetric-table write failures.** Writing symmetric MAU tables against the pipe-0 scope was rejected `INVALID_ARGUMENT` while reads succeeded; the fix targets the device-global scope `0xffff`. (Reads pass, writes fail — a classic silicon-only footgun.)
  9. **Shared-scheduler coupling → two domains.** The central lesson (§7): one strict-priority scheduler starves `qid3/qid2`; dp8 + dp10 give independent schedulers.
  10. **TCP timestamps leaking timing.** The anti-subtraction guarantee is valid only on a no-TCP-timestamp flow (the timestamp option is an independent timing channel). The setup fails closed on any kind-8 timestamp; on silicon `net.ipv4.tcp_timestamps = 0` was set and verified at packet level (0 SYN/SYN-ACK carried the option).
  11. **Missing relay-facing capture.** dp68 is internal, not a tap, so `T0 + J` and release multiplicity were not captured — exactly-once is *inferred* from the ~24.8 ms round trip, not measured.
  12. **Two-pipe vs one-pipe BOR + the stage-recovery wall.** A “faithful” BOR (first OPERATE genuinely *held*, not fail-opened) needs the reservoir ready at SELECT time. The additive one-pipe BOR probe needed **14** ingress stages (13 with aux features off); maximal consolidation reached **13** — still one over budget, the irreducible hold/release core. The shipped fit came from the decision-table flatten (§10), which took RRC 12 → 10 and let BOR fill the freed 2 → **12 in one pass**, so the two-pipe split was not needed. (Bonus TF1 lesson: a register's RegisterActions must share the register's single stage — a read pinned early and a confirm pinned late cannot co-locate.)

---

## 13 13. Experiments and results (E0-E8)

**Plain version.** Native and defended traffic were captured as *paired* runs on the same binary, toggled OFF vs D4, on the physical relay with TCP timestamps off. The numbers below are read straight from the frozen CSV/JSON at commit 5a0fb73.

**What each stage is.** E0 testbed preservation; E1 native baseline; E2 defended READ + SELECT; the SBO/BOR J-independence axis ( $J = 2/6/12$ ); E4 CLRT + JS + MI; E5 READ-vs-SELECT classifier; E7 completion criteria; plus size reconstruction, safety, and compiler-resource evidence.

**Verified results.**

| #                      | Result                      | Value                                                             | Source                           |
|------------------------|-----------------------------|-------------------------------------------------------------------|----------------------------------|
| Size (native)          | READ responses              | <b>100 × [49]</b> , all CRC/checksum-valid                        | size_verdict.csv                 |
| Size (defended READ)   | segments                    | <b>600 × [28,21]</b>                                              | size_verdict.csv                 |
| Size (defended SELECT) | segments                    | <b>500 × [28,21]</b>                                              | size_verdict.csv                 |
| Size (SBO)             | class split                 | <b>90 SELECT + 90 OPERATE</b> , all [28,21]                       | sbo_*.csv                        |
| Size escapes           | 49-byte defended leaks      | <b>0</b>                                                          | VERDICT.json                     |
| Reconstruction         | CRC + IP/TCP checksum valid | <b>1280 / 1280</b>                                                | size_reconstruct.py              |
| CLRT READ              | median / std                | native 1.272 / 1.354 ms → <b>4.001 / 0.022 ms</b>                 | verdict_stats.json               |
| CLRT SELECT            | median / std                | native 2.107 / 2.530 ms → <b>4.001 / 0.021 ms</b>                 | verdict_stats.json               |
| BOR J-independence     | echo – ACK, J=2/6/12        | <b>4.00 ms, std ≈0.026</b> , invariant                            | sbo_j{2,6,12}.csv                |
| MI(class; CLRT)        | common-bins                 | 0.424 bits → <b>0.0018 bits</b> (within perm-null [1e-6, 0.0032]) | verdict_stats.json               |
| Classifier             | READ-vs-SELECT BA           | 0.592 → <b>0.500 (chance)</b>                                     | verdict_stats.json               |
| Safety                 | relay outputs               | <b>all 32 OPEN</b> ; index-6 breaker-close hard-refused           | readbacks/relay_outputs_final.js |
| Resource fit           | pipe / stages               | <b>1 pipe, ≤12 ingress stages</b>                                 | compile logs                     |

**Notes on method.** CLRT stats exclude the cold poll (first transaction after SYN). The MI and JS use pre-declared common bins `linspace(0,12,61)` ms. `scipy.jensenshannon` returns **distance** (=  $\sqrt{\text{divergence}}$ ): native-vs-defended READ distance 0.997, SELECT 0.914 — i.e. the defended distribution is a **replacement** of the native one, and READ-vs-SELECT collapse together after defense (distance 0.68 → 0.043).

## 14 14. How to interpret every figure

- **FIG-0 — overview flow.** Axes: none (block flow). Reads as: DNP3 traffic → classify → timing policy → RRC/BOR queues → CRC split → normalized traffic. Takeaway: the whole defense in one line. Does not prove: anything numeric — it is the map, not the measurement.
- **FIG-1 — CLRT distribution.** X: CLRT (ms); Y: density. Reads as: native READ/SELECT are broad, low; defended collapses to a spike at 4 ms. Takeaway: the timing signature is replaced by a constant. Does not prove: multi-device anonymity.

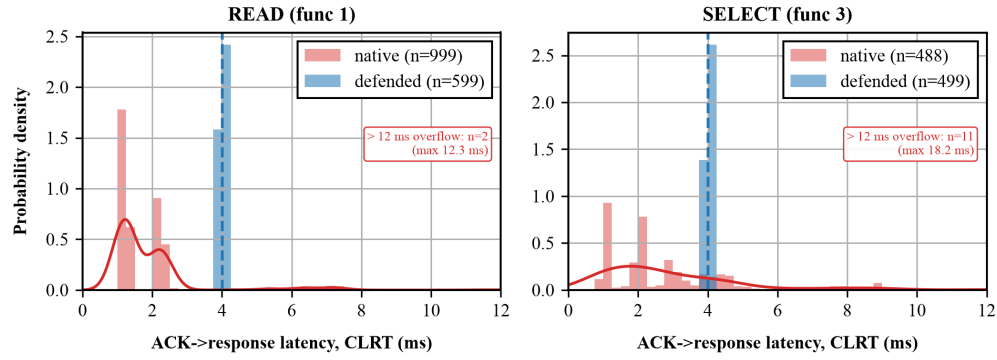


Figure 11: **FIG-1.** CLRT distributions: broad, low-variance-defeating native READ/SELECT curves collapse to a single spike at 4 ms under defense.

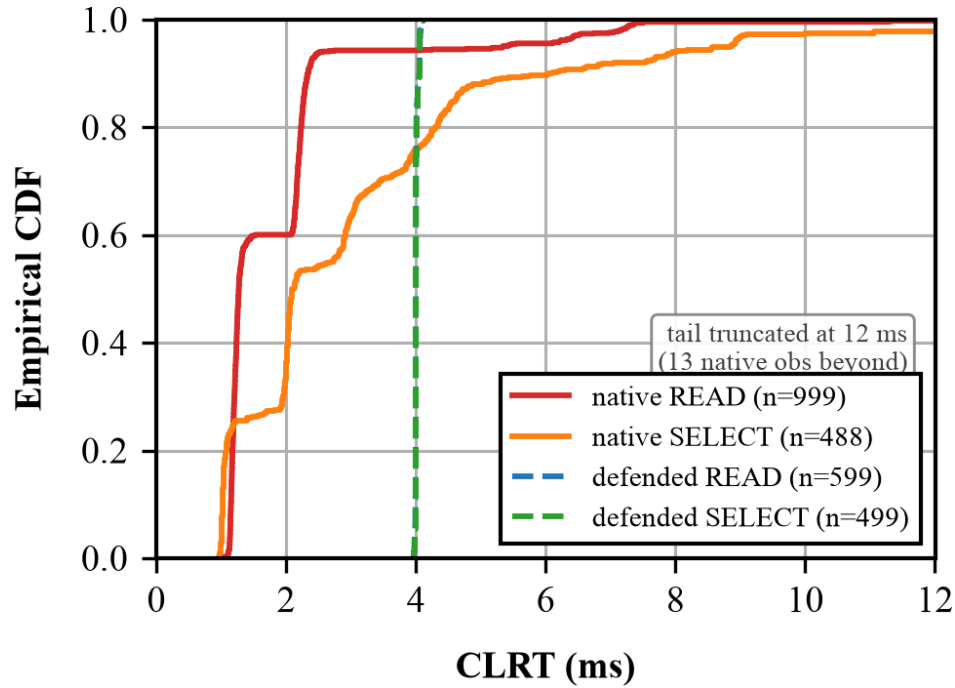


Figure 12: **FIG-2.** CLRT CDF: the defended curve is a near-vertical step at 4.00 ms; native curves have long tails.

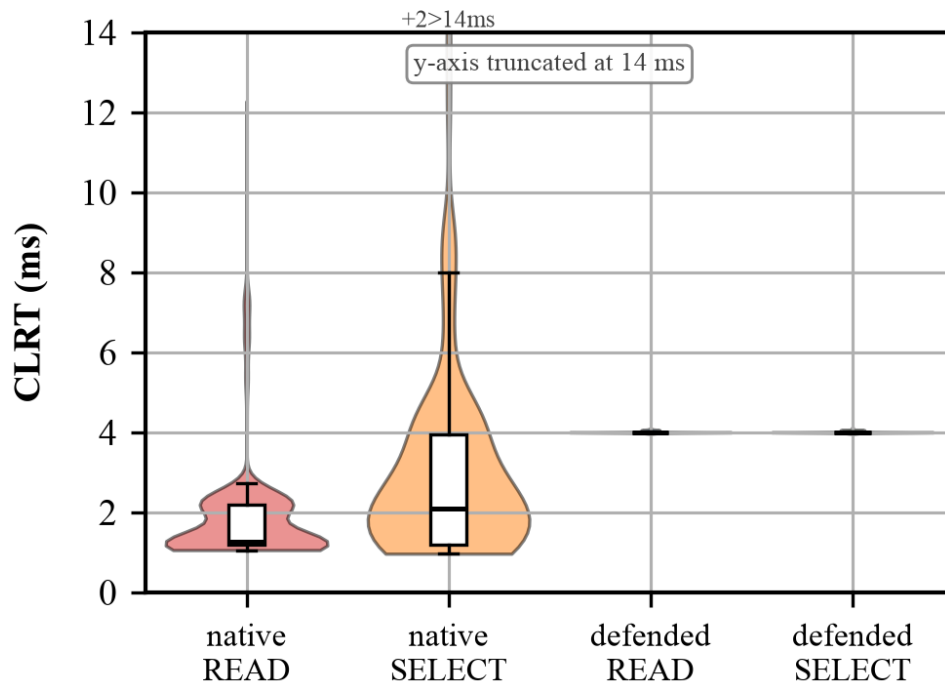


Figure 13: **FIG-3.** CLRT by class  $\times$  mode: wide native violins vs a flat defended line — variance destroyed, READ and SELECT merged.

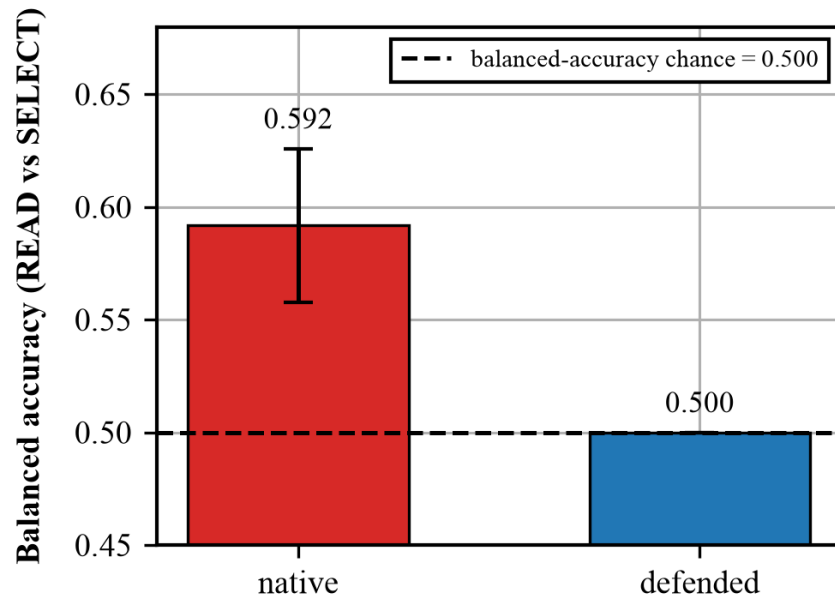


Figure 14: **FIG-6.** Attacker's READ-vs-SELECT classifier balanced accuracy: 0.592 (native)  $\rightarrow$  0.500 = chance (defended).

- **FIG-2 — CLRT CDF.** X: CLRT (ms); Y: cumulative fraction. *Reads as:* defended CDF is a vertical step at 4 ms. *Takeaway:* essentially zero variance. *Verbal line:* “half the native READs answer by 1.3 ms with a long tail; every defended one answers at 4.00 ms.”
- **FIG-3 — CLRT violin.** X: class  $\times$  mode; Y: CLRT (ms). *Reads as:* fat native violins vs a flat defended line. *Takeaway:* variance destroyed, classes merged.
- **FIG-4 — BOR J-independence.** X:  $J \in \{2,6,12\}$  ms; Y: offset (ms). *Reads as:* A and R sit on flat lines; echo – ACK is a constant 4 ms bracket. *Takeaway:* the hidden J does not move the observable. *Does not prove:* the relay-facing T0+J (not captured).
- **FIG-5 — segment size.** X: segment length (B); Y: count. *Reads as:* native mass at 49; defended mass at 28 and 21. *Takeaway:* uniform segmentation, zero 49-byte escapes.
- **FIG-6 — classifier BA.** X: mode; Y: balanced accuracy. *Reads as:* native 0.59  $\rightarrow$  defended 0.50 (chance line). *Takeaway:* the CLRT feature no longer separates READ from SELECT. *Does not prove:* device identification (single device).
- **FIG-7 — testbed topology.** The physical rig and port map (§2).
- **FIG-8 — Tofino pipeline + TM queues.** Ingress dispatch  $\rightarrow$  one meta.outcome  $\rightarrow$  one tbl\_commit; the TM with dp8 (qid7 $\rightarrow$ qid4) and dp10 (qid3,qid2) strict-priority ladders; dp68 seeding the \*\_BLOCK reservoirs. *Takeaway:* where every mechanism physically sits.
- **FIG-9 — BOR timing + anti-subtraction.** Two tracks: internal (T0+J hidden) and master-facing (ACK@T0+A, echo@T0+R); echo – ACK = R – A invariant across J. *Takeaway:* the anti-subtraction property, visually.
- **FIG-10 — RRC/BOR primitive.** The two mechanisms as process flows (recirculate $\rightarrow$ carve $\rightarrow$ hold; SELECT-preseed $\rightarrow$ hold $\rightarrow$ release-once). *Takeaway:* the algorithms, step by step.
- **FIG-11 — DNP3 frame + cut point.** Axes: byte offset (0-49). *Reads as:* link header (10) + block 0 (18) | block 1 (18) + final (3), with the cut at byte 28. *Takeaway:* the split is on a real CRC boundary; join = original. *Verbal line:* “we only ever cut where the standard already put a CRC.”
- **FIG-12 — SBO transaction.** *Reads as:* SELECT pre-seeds the epoch; OPERATE is held; released once at T0+J; ACK/echo anchored to T0; retransmit  $\rightarrow$  dropped. *Takeaway:* the whole control path, including the exactly-once handling. *Does not prove:* relay-facing single delivery (inferred).
- **FIG-13 — two-pass pipeline (Ditto-style, improved).** *Reads as:* classify  $\rightarrow$  strict-priority blocker/hold pairs  $\rightarrow$  loopback (2nd pass)  $\rightarrow$  deadline release + CRC split. *Takeaway:* the architecture at a glance, and how it differs from Ditto (deadline gates, not chaff; splitting, not padding).

(FIG-8 and FIG-10 exist in two renditions: the vector exact-label master (diagram-design SVG) and a papervizagent gpt-image raster alternate. Use the vector master for the manuscript.)

## 15 15. Claim and limitation guide

What is claimed (PASS).

- **Timing (CLRT) normalization** — native 1.27/2.11 ms high-variance  $\rightarrow$  4.001 ms std 0.02.
- **Size + segmentation normalization** — [49]  $\rightarrow$  [28,21], 1280/1280 CRC/checksum-valid, 0 escapes.
- **Formby CLRT feature-suppression** — READ-vs-SELECT MI 0.424  $\rightarrow$  0.0018 bits (within null); classifier BA 0.592  $\rightarrow$  0.500.
- **Testbed unchanged** — native and defended are paired trials on the same binary.
- **Safety** — all 32 relay outputs OPEN throughout; index-6 breaker-close hard-refused; no

actuation.

### What is partial or not claimed.

- **Exactly-once BOR — PARTIAL / not demonstrated.** The relay-facing wire (dp68 internal) was not captured; T0 + J and release multiplicity are *inferred*, not measured. The master issued one OPERATE per transaction — that much is observed — but this is not duplicate-suppression evidence.
- **Multi-device indistinguishability — NOT claimed.** One SEL-751 was tested; its signature is *replaced* by policy, not made indistinguishable from other devices.
- **Byte-identical-to-source — NOT claimed.** No source-frame oracle; only CRC/checksum-valid 49-B reconstruction.
- **A/R offset** — the ~21/25 ms master-facing A/R include ~1 ms of path/capture offset over the configured 20/24 ms; internal deadline vs switch-T0 was not separately instrumented.
- **Classifier split** — transaction-disjoint, not session-disjoint (single capture session).

**The three speaking levels and the likely questions** (30 s / 3 min / detailed, plus “why delay both ACK and response,” “why J,” “why two ports,” “does [28,21] really hide size,” “can the attacker reassemble TCP,” “retransmission,” “why exactly-once is PARTIAL,” “vs software shaping,” “what is novel on Tofino”) are in the companion **MEETING\_REFERENCE.md** — keep that one page in the room.

---

*Authoritative sources (commit 5a0fb73): defense4/size/native\_parity/p4/defense4\_rrc\_bor\_unified12.p and defense4\_crc\_split\_kernel.p4; setups \*\_unified12\_setup.py, \*\_crc\_split\_setup.py, \*\_bor\_twopipe\_setup.py; evidence evidence/E\_FINAL/ (VERDICT.json, verdict\_stats.json, CLAIM\_MATRIX.md, csv/, scripts/, readbacks/); defect ledger RRC\_BOR\_UNIFIED12\_FIXED.md; stage matrix BOR\_STAGE\_RECOVERY\_RESULT.md; two-pipe BOR\_TWO\_PIPE\_FAITHFUL\_RESULT.md.*